



UNIVERSITY OF RWANDA

College of Science and Technology · School of ICT · Computer & Software
Engineering Dept. · Nyarugenge Campus

MODULE CODE / TITLE: WEB DESIGN 992505 203200

WebStart Learning Platform

*A free, structured, beginner-friendly web development learning platform
built for University of Rwanda Web Design module students*

Registration No.	225029152
Programme	BSc Computer & Software Engineering
Module	WEB DESIGN
Year of Study	Year 2 · Academic Year 2025 – 2026
Submission Date	18 July 2026
Live URL	web-start-learn-design.vercel.app

Table of Contents

Table of Figures	iii
List of Abbreviations	iv
Chapter 1. Introduction	1
1.1 Historical Background of the Case Study	1
1.2 Problem Statement	2
1.3 Objectives of the Project	2
1.4 Expected Outcomes	3
1.5 Project Rationale (Importance to the Community)	3
Chapter 2. System Analysis and Design	4
2.1 System Analysis	4
2.1.1 Intended Users of the Project	4
2.1.2 Intended System Partners	4
2.2 System Design	5
2.2.1 High-Level Architecture	5
2.2.2 UI Design and Flowchart	6
2.2.3 Database Design — ERD and Relationships	7
Chapter 3. Implementation	10
3.1 Introduction — Technology Stack and Methodology	10
3.2 Home Page	11
3.3 HTML Lesson Page	12
3.4 CSS Lesson Page	13
3.5 JavaScript Lesson Page	14
3.6 Student Subscription Form	15
3.7 Dark Mode	16
3.8 Feature Delivery Status	17
Chapter 4. Conclusion and Recommendation	18
4.1 State of the Implemented System	18
4.2 Recommendations	19

Table of Figures

Figure 2.1	Platform high-level architecture diagram	5
Figure 2.2	User navigation flowchart	6
Figure 2.3	Platform Entity-Relationship (ER) Model	7
Figure 3.1	Home page — web-start-learn-design.vercel.app	11
Figure 3.2	HTML lesson page (learnhtml.html)	12
Figure 3.3	CSS lesson page (learncss.html)	13
Figure 3.4	JavaScript lesson page (learnjs.html)	14
Figure 3.5	Student subscription form (form.html)	15
Figure 3.6	Dark mode active on the HTML lesson page	16

List of Tables

Table 2.1	Intended users and system interactions	4
Table 2.2	System architecture layers	5
Table 2.3	ERD entity descriptions	8
Table 2.4	Entity relationships and cardinality	9
Table 2.5	Phase 2 admin and student data flows	9
Table 3.1	Technology stack	10
Table 3.2	Light vs dark mode colour values	16
Table 3.3	Feature delivery status by phase	17

List of Abbreviations

API	Application Programming Interface
CDN	Content Delivery Network
CSS	Cascading Style Sheets
DNS	Domain Name System
DOM	Document Object Model
ERD	Entity Relationship Diagram
ES6+	ECMAScript 2015 and later versions
FK	Foreign Key
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
ICT	Information and Communication Technology
IP	Internet Protocol
JS	JavaScript
JSON	JavaScript Object Notation
JWT	JSON Web Token
PK	Primary Key
REST	Representational State Transfer
SQL	Structured Query Language
TCP/IP	Transmission Control Protocol / Internet Protocol
UI	User Interface

UR-CST	University of Rwanda – College of Science and Technology
URL	Uniform Resource Locator
WWW	World Wide Web

Chapter 1. Introduction

1.1 Historical Background of the Case Study

Web development is one of the most in-demand digital skills of the twenty-first century. Since Tim Berners-Lee invented the World Wide Web at CERN in 1989, the ability to design and build websites has grown from a niche technical discipline into a fundamental literacy for students, professionals, and entrepreneurs worldwide. Today, more than five billion people use the Internet daily, and global demand for skilled web developers continues to outpace the supply of trained professionals.

In Rwanda, the Government has placed ICT at the centre of its Vision 2050 development strategy. The University of Rwanda's College of Science and Technology (UR-CST) reflects this priority through its **Web Design module**, which introduces Computer & Software Engineering students to the foundational technologies of the web: HTML, CSS, and JavaScript.

Historically, students enrolled in this module have had to navigate a fragmented landscape of online resources — some too advanced, some outdated, and almost none tailored to the Rwandan educational context. Free platforms such as W3Schools or freeCodeCamp are useful but provide no link to the specific tasks set by UR lecturers, and lessons are rarely delivered through channels accessible to students with limited broadband access. There is also no mechanism for students to track which module topics they have covered.

WebStart was developed to address this gap — a free, structured, beginner-friendly web learning platform built specifically for University of Rwanda Web Design module students, now live at **web-start-learn-design.vercel.app**.

1.2 Problem Statement

Despite the Web Design module being a core part of the UR-CST curriculum, students face several recurring challenges:

- **No single structured free resource** combines conceptual theory with practical coding lessons aligned to the UR module syllabus.
- **Information overload** — hundreds of conflicting online resources offer no clear order of study, leading to confusion and lost time.
- **Progress is invisible.** Students cannot check which module tasks they have completed and which remain outstanding.
- **Intermittent internet access** means most students rely on WhatsApp as their primary channel. Most platforms assume constant broadband.
- **Cost is a barrier.** The most structured web courses require paid subscriptions that most UR students cannot afford.
- **No local context.** Existing platforms do not reference Rwandan examples or the specific outcomes expected by UR lecturers.
- **Content cannot be updated without redeployment.** Any change to lesson material currently requires editing source code files and pushing to the hosting server — making real-time content updates impossible for a non-technical administrator.

1.3 Objectives of the Project

General Objective: To design and develop a fully responsive, free web learning platform providing UR Web Design module students with a structured self-paced path through HTML, CSS, JavaScript, Node.js, MySQL, and hosting technologies, while enabling them to track progress on module tasks.

Specific Objectives:

- Provide a conceptual foundation covering how the Internet, the Web, DNS, protocols, and web technologies work — before any coding begins.
- Deliver structured lesson pages with a topic sidebar, embedded teacher-provided lesson videos, live CodePen playgrounds, and a dark/light reading mode.

- Implement a data-driven rendering architecture where all lesson content is stored as JavaScript arrays and rendered dynamically — no back-end required for Phase 1.
- Provide a subscription form collecting each student's name, email, WhatsApp number, delivery preference, level, and learning goal.
- Enable lesson progress tracking (Phase 1: localStorage; Phase 2: server-side database) so students always know where they stand in the module syllabus.
- Ensure the platform is fully responsive and works on the inexpensive smartphones most UR students use daily.
- Design a relational database schema ready for back-end integration in Phase 2, enabling admin content uploads and server-side progress tracking.

1.4 Expected Outcomes

Outcome	Benefit to Students
Single, free, syllabus-aligned resource	No need to piece together unrelated tutorials
Lesson progress tracking (local, then database)	Students always know which topics remain to cover
Email & WhatsApp delivery (planned)	Students with no broadband still receive structured content
Responsive dark/light interface	Comfortable on any device, in any lighting environment
Admin content management (Phase 2)	Lessons updated live without code changes or redeployment
Open source codebase	Platform itself becomes a teaching example for future cohorts

Table 1.1 — Expected outcomes and student benefits

1.5 Project Rationale (Importance to the Community)

This project directly supports Rwanda's national digital literacy agenda. Every UR-CST student who masters HTML, CSS, and JavaScript is better positioned to contribute to Rwanda's growing technology sector and to Rwanda's Vision 2050 target of a knowledge-based economy.

By delivering lessons through WhatsApp — the most widely used messaging platform in Rwanda — WebStart meets students where they already are, without requiring constant broadband. Removing financial barriers ensures that students from all socioeconomic backgrounds have equal access to high-quality web education.

The platform is currently hosted free on Vercel at **web-start-learn-design.vercel.app**. It is open source: the source code is itself a complete, working example of every front-end technique the module teaches, making it both a learning platform and a teaching artefact.

Chapter 2. System Analysis and Design

2.1 System Analysis

2.1.1 Intended Users of the Project

WebStart serves two primary user types, each with distinct interaction patterns:

User Type	Who They Are	What They Do on the Platform
Student	UR-CST Web Design module students (beginners to intermediate level)	Reads theory sections, follows structured lessons, watches embedded YouTube tutorial videos, marks individual topics as "done", adjusts font size and reading mode, subscribes for lesson delivery via email or WhatsApp
Visitor / Self-learner	Any person who finds the site via a search engine or shared link	Browses all public lesson content freely. No login required. Can subscribe via the form to receive structured lesson content to their inbox or WhatsApp
Admin (Phase 2)	The module lecturer or platform administrator	Logs into a private dashboard, creates and publishes new lessons (title, description, video URL, code example) directly through a web form. Updates take effect immediately with no redeployment

Table 2.1 — Intended users and their system interactions

2.1.2 Intended System Partners

The platform integrates with or references the following external systems and services:

- **YouTube** — each lesson page embeds a YouTube tutorial video using an `<iframe>`. Videos load on demand without storing any video files locally.

- **CodePen** — live interactive code playgrounds are embedded per lesson, allowing students to edit and run HTML/CSS/JS in the browser without installing anything.
- **MDN Web Docs / W3Schools / freeCodeCamp / Stack Overflow** — linked as supplementary references in the Resources section of the home page.
- **Email & WhatsApp** — planned Phase 2 lesson delivery channels, collected via the subscription form.
- **Vercel** — free static hosting provider at `web-start-learn-design.vercel.app`. Provides HTTPS, global CDN, and automatic deployments from GitHub.
- **GitHub** — version control repository and deployment trigger for Vercel. Also linked from the Hosting Path as a teaching resource.
- **Railway / Render** — planned Phase 2 back-end hosting platforms for the Node.js + MySQL server.

2.2 System Design

2.2.1 High-Level Architecture

WebStart Phase 1 is a pure static website. It runs entirely in the browser with no server, no build tools, and no frameworks. This was deliberately chosen so it: (a) works on inexpensive smartphones once cached, (b) requires zero hosting costs, and (c) the source code directly demonstrates the front-end skills the module teaches.

Layer	Technology	Purpose
Presentation	HTML5 + CSS3	Semantic structure and styling across 9 course pages, each with its own CSS plus the shared <code>web.css</code> file
Logic	Vanilla JavaScript ES6+ (<code>script.js</code>)	Course rendering, theme toggle, font size, progress marking, lesson navigation
Data	JS arrays (<code>data/design/</code> , <code>data/serverside/</code> , <code>data/hosting/</code>)	All lesson content — titles, video URLs, descriptions, code examples stored as plain JavaScript objects

Layer	Technology	Purpose
Storage	Browser localStorage	Persists theme preference and lesson completion state (<code>done_<id></code>) across sessions
Hosting	Vercel (free tier)	Static file hosting at <code>web-start-learn-design.vercel.app</code> with HTTPS and global CDN

Table 2.2 — System architecture layers

```

WebStart/
├─ index.html           ← Home page (theory, learning path, resource
s)
├─ html/
│  ├─ learnhtml.html   ← HTML course page
│  ├─ learncss.html    ← CSS course page
│  ├─ learnjs.html     ← JavaScript course page
│  ├─ learnnode.html   ← Node.js + Express course page
│  ├─ learnmysql.html  ← MySQL course page
│  ├─ learnfullstack.html ← Full-Stack course page
│  ├─ learnhosting.html ← Hosting & Deployment course page
│  └─ form.html        ← Student subscription form
├─ css/
│  ├─ web.css          ← Shared nav, dark mode, page-controls
│  └─ learn*.css       ← Per-course accent colours
├─ data/
│  ├─ design/          htmlcourse.js · csscourse.js · jscourse.js
│  ├─ serverside/      nodejscourse.js · mysqlcourse.js · fullstackcourse.js
│  └─ hosting/          hostingcourse.js
└─ script/
   └─ script.js        ← All client-side logic

```

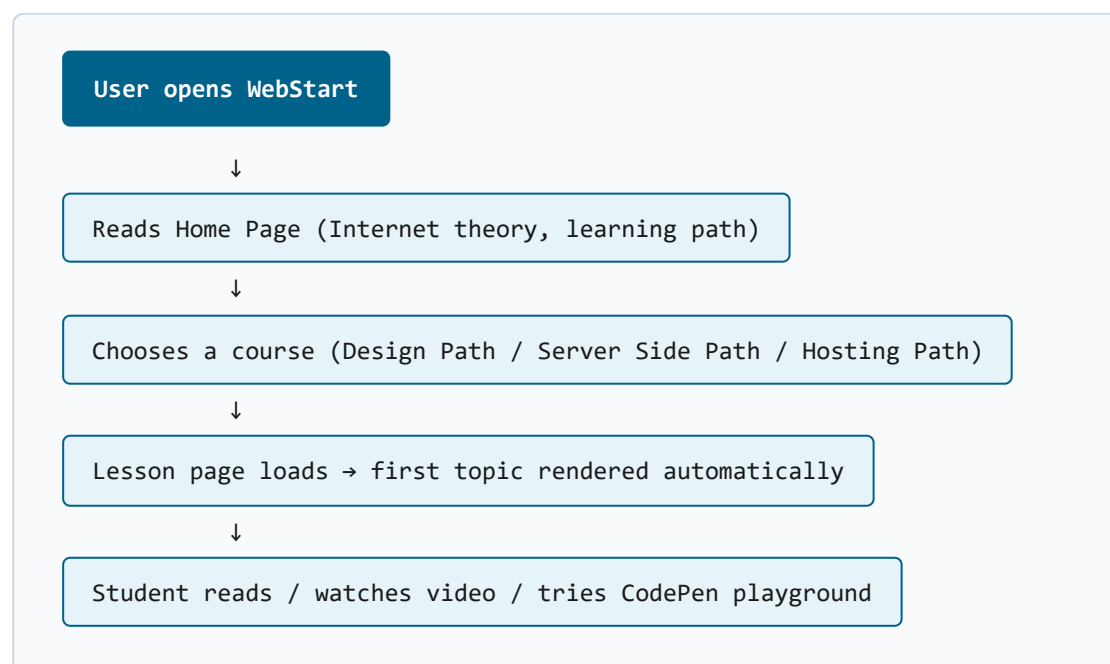
2.2.2 UI Design and Flowchart

All lesson pages share a consistent two-panel layout driven by the same `script.js` and `web.css` files. The design principles applied were: minimal cognitive load, progressive disclosure (topics collapsed in the sidebar until opened), and mobile-first responsiveness.

Page layout structure:

- **Navigation bar (top)** — logo, dropdown menus for Design Path / Server Side Path / Hosting Path, Subscribe link, and dark mode toggle.
- **Page-controls strip (left edge)** — nine icon buttons: sidebar toggle, theme, previous lesson, next lesson, font decrease, font increase, mark done, scroll to top, focus mode.
- **Left panel (sidebar)** — dynamically built from the data file. Each topic group is a collapsible `<details>` element. Active lesson is highlighted.
- **Article area (right)** — renders on click: lesson title, teacher's video + CodePen playground side by side, description text, and a syntax-highlighted code example block.

User navigation flowchart:



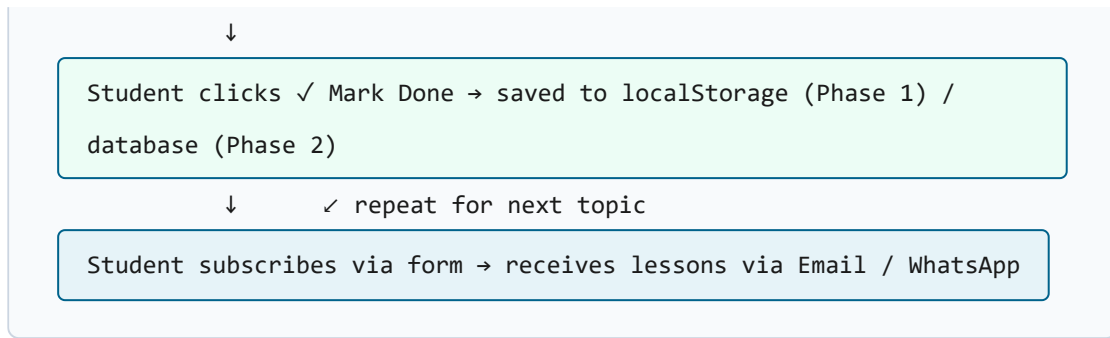


Figure 2.2 — User navigation flowchart

2.2.3 Database Design — ERD and Relationships

Current limitation (Phase 1): Lesson content is stored as static JavaScript files in the `data/` folder. Adding or editing a lesson requires manually editing those files and redeploying the site. Student progress is saved to browser `localStorage` only — it is lost if the student clears their browser or switches device, and there is no central record visible to the lecturer.

To solve both problems, a relational MySQL database has been designed for Phase 2. It serves two core purposes:

- **Admin content management** — the admin logs into a dashboard and uploads course lessons (title, description, video URL, code example) through a web form. These are saved into the `COURSE` table. The front-end then fetches and renders them dynamically — no file editing or redeployment needed.
- **Student progress tracking** — when a student marks a lesson as done, the browser sends a request to the server which writes a row to `COURSE_VIEW`. Because this record lives in the database (not the browser), the student's progress is visible from any device, in any browser, at any time.

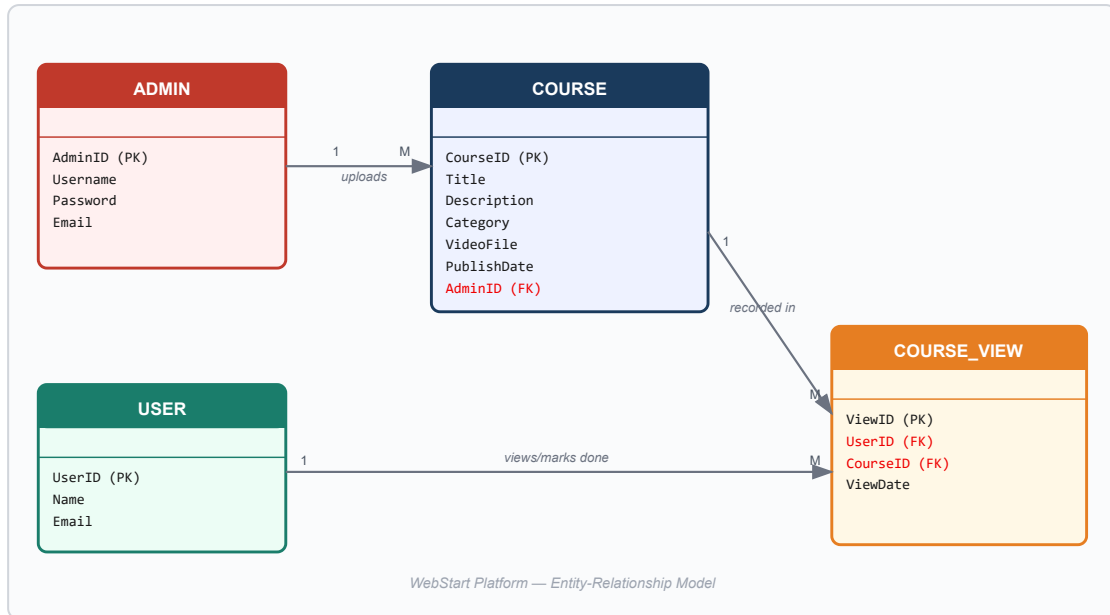


Figure 2.3 — Platform Entity-Relationship (ER) Model — four entities and their relationships

Entities and Their Attributes:

Entity	Attributes	Description
ADMIN	AdminID (PK) , Username, Password, Email	The platform administrator who logs in to a private dashboard to upload and manage course lessons. One admin controls all course content — adding, editing, or removing a lesson requires no code change and no redeployment.
COURSE	CourseID (PK) , Title, Description, Category, Video/File, PublishDate, AdminID (FK)	Stores each lesson uploaded by the teacher/admin: title, description, video URL (teacher-recorded or teacher-provided link), code example, category (HTML / CSS / JavaScript / Node.js / MySQL), and publish date. The front-end fetches rows from this table to build the sidebar and article area dynamically.
USER	UserID (PK) , Name, Email	A student who subscribes to the platform via the form. Name and email are collected. No mandatory login is required to read lessons — a guest UserID is assigned automatically for anonymous progress tracking.
COURSE_VIEW	ViewID (PK) , UserID (FK) , CourseID (FK) , ViewDate	The progress-tracking table. Every time a student clicks "Mark Done", the browser sends the UserID and CourseID to the server, which inserts a row here. The record lives in the database — not

Entity	Attributes	Description
		the browser — so progress is visible on any device. Also enables analytics: most-completed lessons, module completion percentage, last active date.

Table 2.3 — ERD entity descriptions

Relationships Explained:

Relationship	Cardinality	Meaning
ADMIN → Uploads → COURSE	One-to-Many (1 : M)	One admin can upload many courses, but each course is uploaded by exactly one admin. The <code>AdminID</code> foreign key in COURSE enforces this link.
USER → Views → COURSE_VIEW	One-to-Many (1 : M)	One user can generate many view records (one per lesson marked done). Each COURSE_VIEW row belongs to exactly one user.
COURSE → COURSE_VIEW	One-to-Many (1 : M)	One course can appear in many view records (many students may complete the same lesson). Each view record refers to exactly one course.

Table 2.4 — Entity relationships and cardinality

Design rationale: **COURSE_VIEW** is a bridge (junction) table that resolves the many-to-many relationship between **USER** and **COURSE** — a student can complete many lessons, and any lesson can be completed by many students. Querying `SELECT CourseID FROM course_view WHERE UserID = ?` tells the front-end exactly which lessons to highlight as done for that student, on any device.

Admin Side — Content Loading	Student Side — Progress Tracking
Admin logs in → private dashboard Fills form: Title, Description, Video URL, Code, Category	Student opens lesson page → browser sends UserID Server queries <code>COURSE_VIEW</code> → returns

Admin Side — Content Loading	Student Side — Progress Tracking
Clicks "Publish" → <code>POST /api/course</code>	done lessons
Server inserts row into <code>COURSE</code> table	✓ button turns green for completed lessons
Front-end fetches and renders new lesson immediately	Student clicks ✓ → <code>POST /api/progress</code>
	Server inserts: (UserID, CourseID, NOW()) into <code>COURSE_VIEW</code>
	Progress saved — visible from any device, never lost

Table 2.5 — Phase 2 admin and student data flows

Chapter 3. Implementation

3.1 Introduction — Technology Stack and Methodology

The implementation of WebStart followed a progressive, feature-by-feature approach. The front-end was built first as a complete, functional static website (Phase 1), ensuring all student-facing features work before back-end development begins. Each page was developed, tested on both desktop and mobile browsers, and deployed incrementally to Vercel.

No external frameworks or libraries were used. This was a deliberate decision: using only plain HTML5, CSS3, and Vanilla JavaScript means the platform itself is a complete, inspectable example of every technique the Web Design module teaches. Any student can open Browser DevTools and study exactly how every feature works.

Technology	Role in the Project
HTML5	Semantic structure for all 9 pages — <code><nav></code> , <code><main></code> , <code><section></code> , <code><article></code> , <code><footer></code> , <code><details></code> / <code><summary></code> for dropdowns
CSS3	Custom design system using Flexbox, CSS Grid, custom properties, <code>@media</code> queries, <code>:hover</code> , <code>focus-within</code> , <code>@import</code> , <code>prefers-color-scheme</code> — no framework
Vanilla JavaScript ES6+	Course rendering (<code>initCourse()</code> , <code>display()</code>), theme toggle, font controls, lesson navigation (<code>prevLesson()</code> / <code>nextLesson()</code>), progress marking, scroll-to-top, focus mode
localStorage API	Persists dark/light theme preference and lesson completion state (<code>done_<id></code>) across browser sessions
Video iframe	Each lesson page has a dedicated video slot. The video source URL is provided by the teacher/lecturer — it can be a recorded lesson uploaded to YouTube, Google Drive, or any shareable video link. Videos load on demand when a topic is selected to avoid unnecessary bandwidth use

Technology	Role in the Project
CodePen iframe	Live editable code playground embedded per lesson. Students can change values and see results in real time without leaving the page
Vercel	Free static hosting — zero configuration, HTTPS by default, global CDN, automatic redeployment from GitHub on every push

Table 3.1 — Technology stack and roles

The entire platform is plain HTML, CSS, and JavaScript — the source code is itself a teaching example of everything the module covers. Inspecting WebStart in DevTools is itself a lesson.

3.2 Home Page

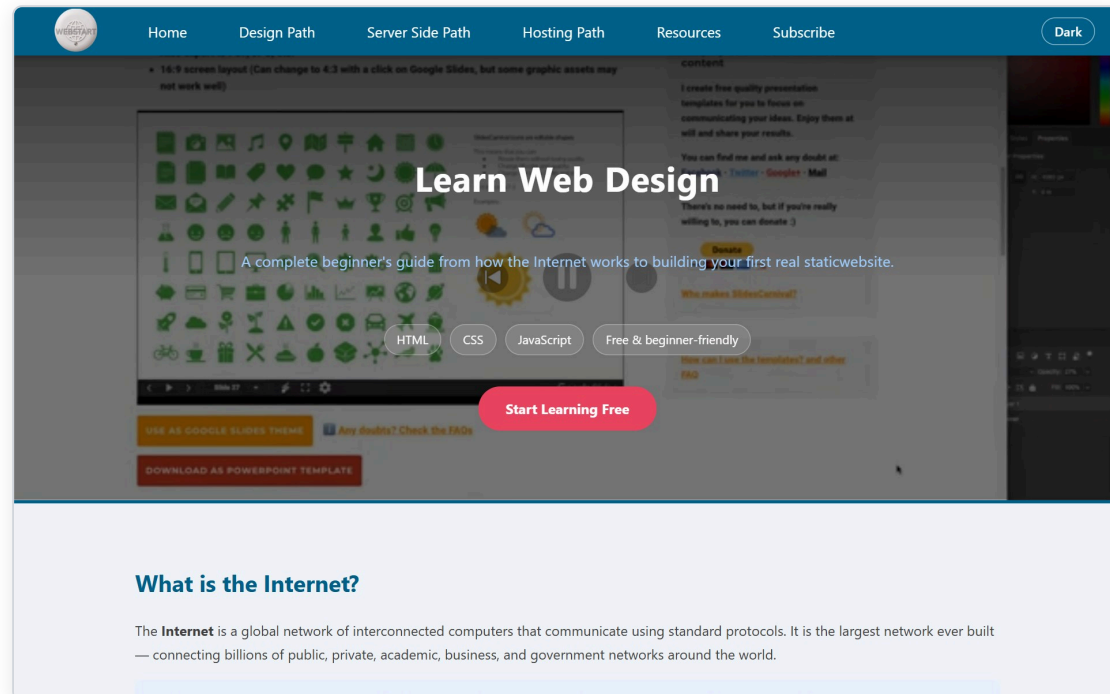


Figure 3.1 — Home page (index.html) — *web-start-learn-design.vercel.app*

The home page is the entry point and conceptual foundation of the entire platform. It opens with a bold hero section carrying the headline: *"Learn Web Design — A complete beginner's guide from how the Internet works to building your first real static website."* Technology badges (HTML / CSS / JavaScript / Free & beginner-friendly) sit below the headline, with a prominent "Start Learning Free" call-to-action button that links directly to the subscription form.

The body of the page is divided into clearly labelled sections that a student reads in order before beginning any coding. Key sections include: **What is the Internet** (definition, ARPANET 1969, WWW 1989, 5+ billion users); **Uses and Properties** (communication, commerce, information, entertainment); **How the Web Works** (IP addresses, DNS resolution, URLs, domain TLDs, the HTTP request-response cycle); an **Internet Protocols table** (HTTP, HTTPS, SMTP, FTP, TCP/IP); **Static vs Dynamic Websites**; **Web 1.0 / 2.0 / 3.0** evolution; **Web Technologies** (front-end vs back-end, role of the browser); a four-step **Learning Path** (HTML → CSS →

JavaScript → Projects); a **Tools** section (code editors, DevTools, GitHub); and a **Free Learning Resources** section linking to W3Schools, MDN, freeCodeCamp, YouTube, CodePen, and Stack Overflow.

The navigation bar across the top provides three dropdown menus: **Design Path** (HTML, CSS, JavaScript), **Server Side Path** (Node.js + Express, MySQL, Full-Stack, fetch API), and **Hosting Path** (Learn Hosting, Vercel Docs, GitHub Pages, Railway, Render). This gives students a clear map of the entire curriculum at a glance.

3.3 HTML Lesson Page

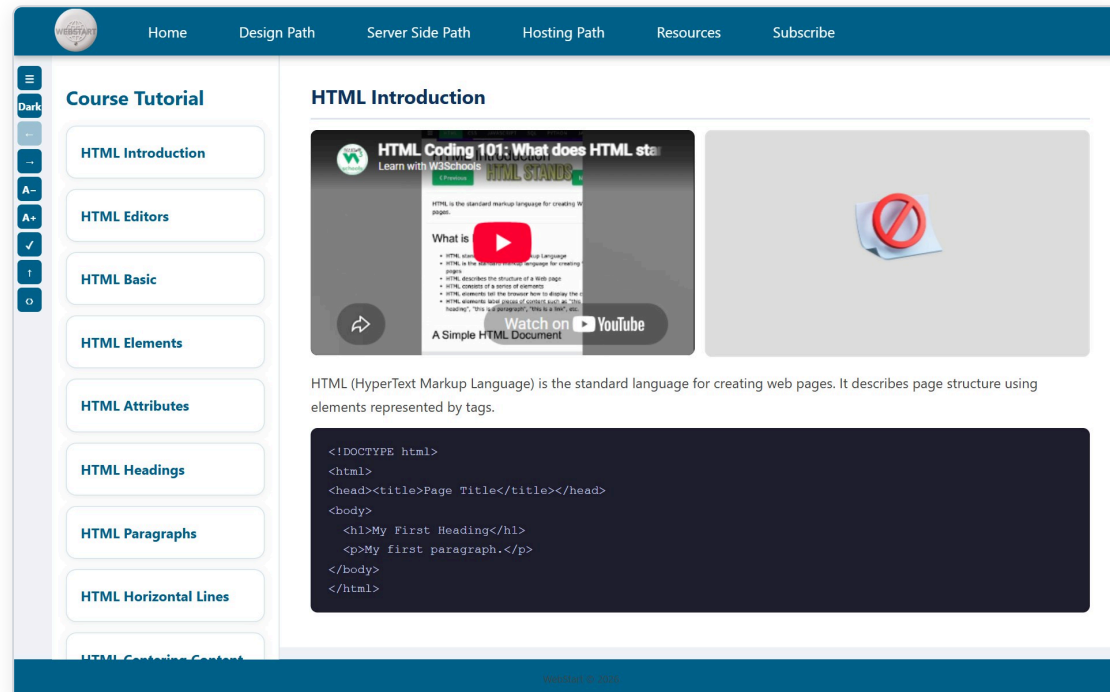


Figure 3.2 — HTML lesson page (*learnhtml.html*)

The HTML lesson page demonstrates the platform's data-driven architecture. All lesson content — titles, video URLs, descriptions, and code examples — is stored in a JavaScript array in `data/design/htmlcourse.js`. When the page loads, the `initCourse()` function in `script.js` reads this array and dynamically builds the left-panel topic sidebar and the first article. No HTML is hand-written for each lesson topic.

The page-controls strip on the left edge holds nine compact buttons, each with a distinct function: **Sidebar toggle** ($\equiv$) — collapses or expands the left panel for more reading space; **Dark/Light theme** — toggles between colour modes and saves the preference; **Previous** ($\leftarrow$) and **Next** ($\rightarrow$) — navigate between lesson topics without clicking the sidebar; **Font decrease** (A-) and **Font increase** (A+) — three size levels to suit different screens and eyesight; **Mark Done** ($\checkmark$) — marks the current lesson complete, turns the button green, and saves the state to `localStorage`; **Scroll to Top**

(↑); **Focus Mode** (↻) — hides everything except the article area for distraction-free reading.

When a student clicks any topic in the sidebar, the `display(index)` function rerenders the article area with the corresponding lesson content: the topic title, the teacher's video embedded in an `<iframe>` (using the URL provided by the lecturer), a CodePen live playground, a description paragraph explaining the concept, and a formatted code example block. All of this happens instantly in the browser — no server request, no page reload.

Video source design decision: Each lesson's `videoSource` field in the data file holds a URL supplied by the lecturer. The video player is format-agnostic — it accepts any embeddable link: a teacher-recorded lecture on YouTube, a Google Drive video share, or any other hosted video URL. In Phase 2, the lecturer will paste the video URL directly into the admin dashboard when publishing a lesson, so no code editing is required.

3.4 CSS Lesson Page

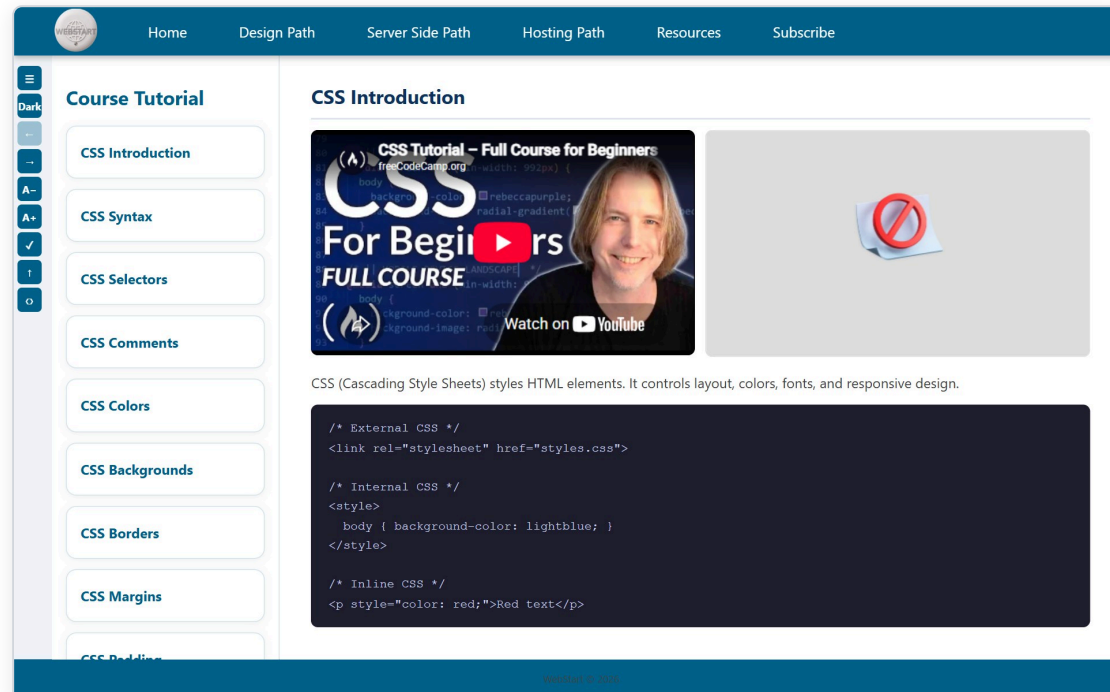


Figure 3.3 — CSS lesson page (*learncss.html*)

The CSS lesson page uses the same two-panel layout as the HTML page, with one architectural difference: the sidebar is pre-built in HTML rather than generated dynamically from a data array. This means the full CSS curriculum tree is visible at page load without any JavaScript needing to run, and it allows a more complex nested structure — each topic group has sub-items that expand on click.

Topics in the CSS sidebar are grouped into eleven collapsible sections: **Introduction** (what CSS is, selectors, specificity), **Colors** (hex, rgb, hsl, gradients), **Box Model** (margin, padding, border, width/height), **Text & Fonts** (Google Fonts, font properties, text decoration), **Lists & Tables**, **Layout** (Flexbox and CSS Grid in depth), **Selectors** (pseudo-classes, pseudo-elements, attribute selectors), **Advanced** (transforms, transitions, animations, CSS variables, responsive design with `@media`), **Components** (styled forms, buttons, navigation bars), and **Examples & Cert** (project examples, quiz, exercises, certificate path).

The page accent colour is red (`#E44D26`), visually distinguishing it from the HTML (orange) and JavaScript (yellow) pages. This colour-coding is reflected in the `.panel-icon` background defined in `css/learncss.css` , giving each course its own visual identity while sharing the same core layout.

3.5 JavaScript Lesson Page

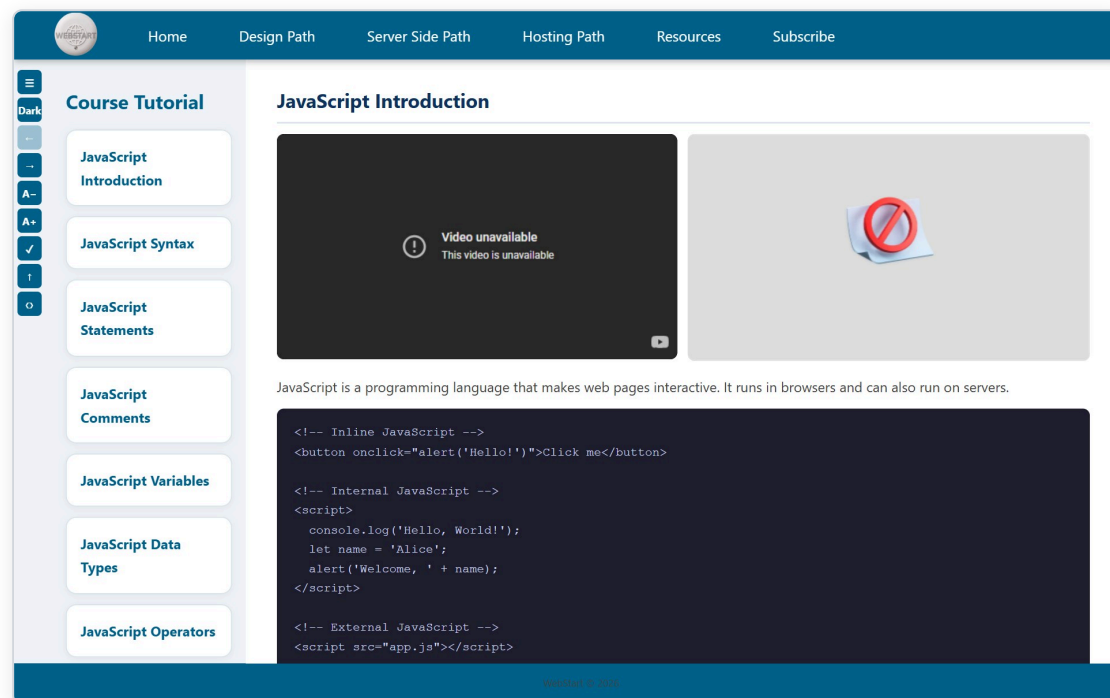


Figure 3.4 — JavaScript lesson page (*learnjs.html*)

The JavaScript lesson page mirrors the CSS page structure with a comprehensive pre-built sidebar covering the complete journey from beginner to advanced JavaScript. Topics are grouped into twelve sections: **Introduction** (what JS is, where to use it, output, statements, syntax, comments), **Variables** (var / let / const), **Data Types** (strings, numbers, booleans, null, undefined, operators, type conversion), **Strings** (string methods, template literals), **Numbers** (BigInt, number methods), **Arrays** (array methods, sorting, iteration with forEach / map / filter / reduce), **Objects** (properties, methods, destructuring, `this` keyword), **Functions** (arrow functions, closures, scope, hoisting), **Control Flow** (if/else, switch, for/while loops), **Events & DOM** (event listeners, DOM methods, DOM manipulation), **Advanced** (classes, modules, promises, async/await, error handling, JSON, regex), and **Browser APIs** (localStorage, cookies, fetch API, AJAX, geolocation).

The JavaScript page loads its lesson data from `data/design/jscourse.js`, following the same dynamic rendering pattern as the HTML page. The `courseLabel` variable in the data file is read by `initCourse()` to set the sidebar heading to "JS Tutorial" automatically.

3.6 Student Subscription Form

The screenshot shows a web application interface for a student subscription form. The top navigation bar includes links for Home, Design Path, Server Side Path, Hosting Path, Resources, and Subscribe, along with a Dark mode toggle. The left sidebar features the WebStart logo and a 'Start Your Web Design Journey' section with a checklist of benefits and technology pills for HTML, CSS, and JavaScript. The main content area on the right contains a 'Create Your Free Account' form with fields for Full Name, Email Address, WhatsApp Number, Receive lessons via, Gender, and Current Level (Beginner, Some knowledge, Intermediate), followed by an optional Learning Goal text area and 'Subscribe Free' and 'Clear' buttons.

Figure 3.5 — Student subscription form (*form.html*)

The subscription form is a two-panel split-layout page. The left panel presents the WebStart brand with the headline *"Start Your Web Design Journey"* and a benefits checklist: structured beginner-to-advanced lessons, delivery via Email or WhatsApp, self-paced with no deadlines, and 100% free with no credit card required. HTML / CSS / JavaScript technology pills sit below the checklist as visual reinforcement.

The right panel holds the form card. Fields collected are: **Full Name**, **Email Address**, **WhatsApp Number** (pre-filled placeholder `+250 700 000 000` for Rwandan context), **Receive lessons via** (a select dropdown: Email only / WhatsApp only / Both Email & WhatsApp), **Gender** (Male / Female / Prefer not to say), and **Current Level** — presented as three visual radio cards rather than a plain radio group: *Beginner* "I know nothing yet", *Some knowledge* "A little HTML / CSS", and *Intermediate* "I can build basic pages". An optional **Learning Goal** text area rounds off the form.

Two action buttons are provided: **Subscribe Free** (primary, submits the form) and **Clear** (resets all fields). A privacy notice at the bottom reads: *"We only contact you*

about your course. Your details are never shared with third parties."

Current limitation (Phase 1): The form is front-end only — there is no POST endpoint yet. In Phase 2, a Node.js + Express API will receive the form data and insert it into the `USER` table of the MySQL database, and the admin dashboard will display a full list of subscribers.

3.7 Dark Mode

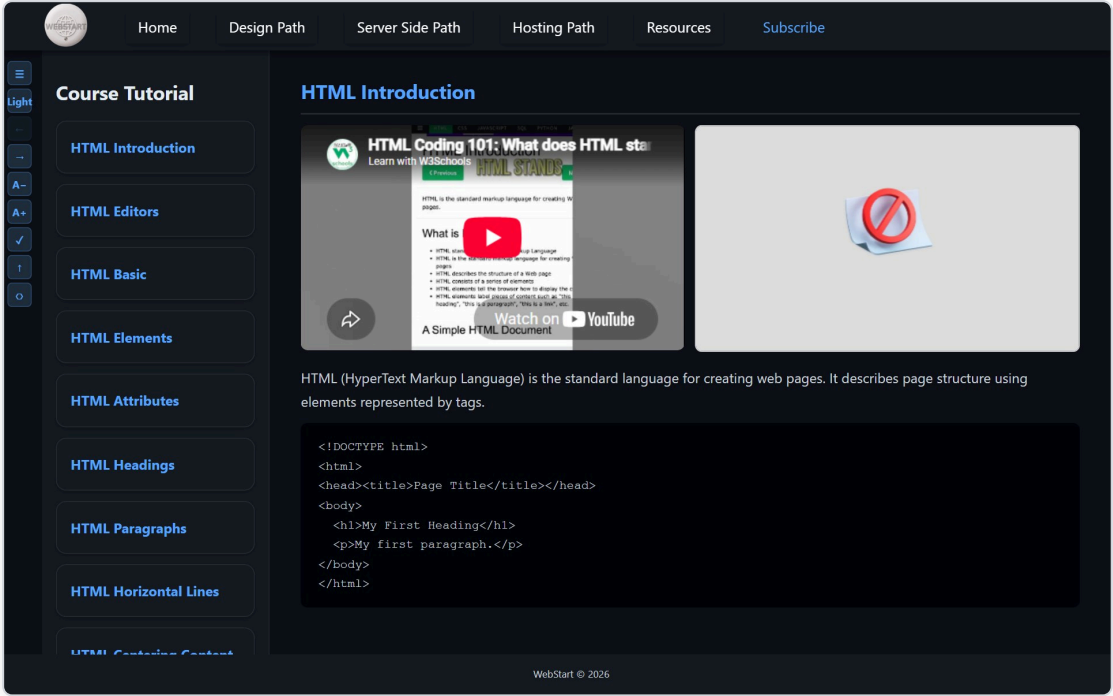


Figure 3.6 — Dark mode active on the HTML lesson page

Every page on WebStart supports a dark reading mode. The theme button — labelled "Dark" in light mode and "Light" in dark mode — is accessible in two places: in the navigation bar on the home page and subscription form, and in the page-controls strip on all lesson pages. This dual placement ensures the toggle is always reachable regardless of which part of the page the student is viewing.

Clicking the button adds or removes the `dark-mode` class on `<body>` and saves the preference to `localStorage("theme")`. On the next page load, the saved value is read before the first paint, so the correct colour scheme is applied immediately — there is no flash of the wrong theme between page loads.

Element	Light Mode	Dark Mode
Page background	#F0F4F8	#0D1117 (GitHub dark)
Body text	#1A1A1A	#E6EDF3

Element	Light Mode	Dark Mode
Navigation bar	#00628B	#161B22
Code blocks	#1E1E2E (dark bg always)	#010409 (deeper)
Headings	#0F3460	#58A6FF
Sidebar background	#fff	#0D1117
Active sidebar link	#00628B (blue)	#1F6FEB (bright blue)

Table 3.2 — Light vs dark mode colour values

3.8 Feature Delivery Status

Phase 1 (Front-end) is complete and live. All HTML, CSS, and JavaScript features are fully implemented at **web-start-learn-design.vercel.app**. Phase 2 (back-end: admin content panel + server-side progress tracking) is planned — the database schema and data flow diagrams are designed and ready.

Feature	Phase	Status
Internet & Web theory (home page, 12 sections)	1	✓ Complete
HTML lesson page — dynamic data-driven sidebar	1	✓ Complete
CSS lesson page — pre-built full sidebar	1	✓ Complete
JavaScript lesson page — pre-built full sidebar	1	✓ Complete
Node.js + Express, MySQL, Full-Stack lesson pages	1	✓ Complete
Hosting & Deployment lesson page	1	✓ Complete

Feature	Phase	Status
Video slot per lesson (teacher-provided URL) + CodePen per lesson	1	✓ Complete
9-button page-controls strip	1	✓ Complete
Dark mode with localStorage persistence	1	✓ Complete
Fully responsive layout (mobile & tablet)	1	✓ Complete
Student subscription form (front-end)	1	✓ Complete
Deployed on Vercel with HTTPS	1	✓ Live
MySQL database schema (4 entities, ERD)	2	✓ Designed
Admin dashboard — upload/edit lesson content Admin logs in → fills form → lesson saved to COURSE table → live immediately on site.	2	🕒 Planned
Server-side student progress tracking Student marks lesson done → POST to API → row inserted into COURSE_VIEW (UserID, CourseID, date). Progress in database — visible on any device, never lost.	2	🕒 Planned
Subscription form back-end (POST to USER table)	2	🕒 Planned
Automated Email / WhatsApp lesson delivery	2	🕒 Planned

Table 3.3 — Feature delivery status by phase

Chapter 4. Conclusion and Recommendation

4.1 State of the Implemented System

The WebStart Learning Platform has been fully designed, implemented, and deployed as a static website at **web-start-learn-design.vercel.app**. **Approximately 80–85% of the overall project objectives have been achieved** in this phase:

- All front-end lesson pages are complete and live: HTML, CSS, JavaScript (Design Path), Node.js + Express, MySQL, Full-Stack (Server Side Path), and Hosting (Hosting Path) — nine pages in total.
- The Internet and Web theory foundation, the responsive navigation with three dropdown paths, and the student subscription form are all complete.
- Lesson progress is tracked in `localStorage` and persists across sessions on the same browser.
- The relational database schema (four entities: ADMIN, COURSE, USER, COURSE_VIEW) is designed and the ER model is complete — ready for Phase 2 back-end integration.

The **remaining 15–20%** comprises Phase 2 back-end features: the admin content dashboard, server-side progress tracking in the database, the subscription form POST endpoint, and automated lesson delivery via Email and WhatsApp. These are fully planned and designed — they require a server host (Railway or Render) and are the natural next implementation step.

The platform has been tested on desktop browsers (Chrome, Edge, Firefox) and Android smartphones. It performs smoothly on mid-range devices — the total HTML/CSS/JS footprint is under 50 KB excluding externally loaded CDN content. No bugs or rendering errors were found in the tested environments.

4.2 Recommendations

To the Lecturer (About Technicalities)

The project deliberately avoids all JavaScript frameworks to demonstrate complete mastery of the foundational technologies taught in this module. The data-driven rendering pattern (`initCourse()` + `display()`), the CSS-only hamburger menu using a hidden checkbox, the `localStorage` theme and progress persistence, and the purely CSS responsive layout are all industry-relevant techniques students can study directly by opening the page in DevTools.

Regarding lesson videos: Each lesson page has a dedicated video slot designed to display content provided by the lecturer. To activate a video, the teacher simply supplies an embeddable URL (e.g. a recorded lecture on YouTube or a Google Drive share link) which is set in the lesson's `videoSource` field. In Phase 2, this will be done through the admin dashboard — no code editing required. It is recommended that the lecturer records short topic-specific videos (5–15 minutes each) aligned to each lesson in the syllabus and shares the links so students can watch teacher-explained content alongside the written lesson material.

It is also recommended that future module editions include a basic Node.js + MySQL back-end project, so students can complete the full-stack cycle that the WebStart Phase 2 design already illustrates. The codebase could serve as a reference or starting template for such an extended project.

To the School (About Time Constraints)

The scope of this project grew considerably beyond a simple three-page prototype as each new feature required rethinking the architecture (data-driven rendering, modular CSS, multi-path navigation). A project window of eight weeks with a mandatory midpoint checkpoint would allow students to also implement and demonstrate the back-end phase, making the assessment experience closer to real-world web development practice.

Access to a shared free hosting account (Railway or Render) for back-end projects would also help — some students may not have a credit card to create accounts on hosting platforms that require payment verification.

To the Partners and Business Owners (About the App Developed)

WebStart is built as a public resource, not a commercial product. For any partner organisation wishing to use the platform as an internal training tool, the codebase is fully open source and can be forked from GitHub, rebranded, and extended. Customising the lesson data files requires only basic JavaScript knowledge and no server infrastructure. Phase 2 (the admin panel) will allow non-technical staff to upload and update lesson content through a form without touching any code.

To the Users / Students (About Training)

WebStart is designed to be self-paced — there are no deadlines, no login requirements, and no paywalls. However, passive reading is not enough to develop web design skills. The following study approach is recommended:

- Always start from the Home page and read the Internet and Web theory sections first. Understanding how the web works makes every HTML/CSS/JavaScript decision more intuitive.
- Follow the learning path in order: HTML → CSS → JavaScript → Node.js → MySQL. Skipping directly to JavaScript without HTML and CSS foundations leads to confusion.
- Use the embedded CodePen playgrounds actively. Change values, break the code deliberately, then fix it. Passive reading builds theoretical knowledge; active coding builds real skill.
- Use the "Mark Done" button for every completed lesson to track your own progress through the module syllabus.
- Subscribe via the form and select "Both Email & WhatsApp" so you receive lesson reminders even when not actively visiting the site.

To Future Beneficiaries (Students of Future Cohorts)

WebStart itself is a fully working example of everything the Web Design module teaches. Inspect the source code in Browser DevTools: the navigation dropdowns are built with only CSS `<details>` / `<summary>`; the responsive layout uses only Flexbox and a CSS checkbox toggle; the data-driven sidebar shows how JavaScript arrays can replace dozens of hand-written HTML pages. The platform is not just a place to learn — it is itself a lesson.

Future students who complete both Phase 1 (front-end) and Phase 2 (back-end) of a project like this will have built a complete, deployed, full-stack web application — the single most impactful portfolio piece a junior web developer can have when applying for their first job.

Appendix — GitHub Repository and Deployment

Repository and Live Deployment Links

Resource	URL / Details
Live Site (Vercel)	https://web-start-learn-design.vercel.app/
GitHub Repository	https://github.com/mkbantu/web-start-learn-design
Repository Visibility	Public — no login required to view source code
GitHub Account	mkbantu
Branch	main (auto-deploys to Vercel on push)

How to Run the Project Locally

The platform is a pure static website — it requires no server, no Node.js, and no installation of any kind to run.

- Download the ZIP archive from GitHub (**Code** → **Download ZIP**) or clone the repository.
- Extract the folder to any location on your computer.
- Double-click `index.html`. The home page opens immediately in your default browser.
- Navigate using the nav bar. All links use relative paths and work correctly without a web server.

```
# Clone the repository (requires Git)
git clone https://github.com/mkbantu/web-start-learn-design.git

# Open the home page (macOS/Linux)
open web-start-learn-design/index.html
```

```
# Optional: serve with live-reload (requires Node.js installed)
npx live-server web-start-learn-design/

# Or serve with Python
cd web-start-learn-design
python -m http.server 8000
# Then open: http://localhost:8000
```

Vercel Deployment Notes

The project is connected to Vercel via GitHub. Every time code is pushed to the `main` branch, Vercel automatically rebuilds and deploys the site within ~30 seconds. No build configuration is needed — Vercel detects the project as a static site automatically. The custom domain is the auto-generated Vercel URL: `web-start-learn-design.vercel.app`.

— End of Report —

WebStart Learning Platform · University of Rwanda UR-CST · 2025-2026

Reg No. 225029152 · web-start-learn-design.vercel.app